

Inteligência Artificial

Licenciatura em Engenharia Informática

Relatório do Trabalho Prático

Pesquisa em Espaço de Estados



UNIVERSIDADE DE ÉVORA

Miguel Pombeiro, 57829 | Miguel Rocha, 58501

Departamento de Informática
Universidade de Évora
Abril 2026

Índice

1	Estados	2
1.1	Exemplo 1	2
1.2	Exemplo 2	3
1.3	Exemplo 3	3
1.4	Exemplo 4	4
2	Operadores Transição Estados	5
2.1	Predicados Auxiliares	5
3	Algoritmo Pesquisa Não Informada	6
3.1	Nós Visitados	7
3.2	Nós em Memória	8
3.3	Comparação de Resultados	8
4	Conclusões Pesquisa Não Informada	9
4.1	Exemplo 1	9
4.2	Exemplo 2	10
4.3	Exemplo 3	10
4.4	Exemplo 4	10
5	Heurísticas	11
5.1	Heurística 1	11
5.2	Heurística 2	11
5.3	Predicados Auxiliares	12
6	Algoritmos Pesquisa Informada	13
7	Conclusões Pesquisa Informada	13
7.1	Heurística 1	13
7.2	Heurística 2	16
	Instruções para Correr o Problema	19

1 Estados

A representação dos estados em Prolog inclui:

- O predicado $a(L, C)$, que representa a posição atual do agente (em que L representa a linha e C a coluna).
- O predicado $m(L, C)$ representa a posição da máquina.
- O predicado $s(L, C)$ representa a posição de saída.
- Uma lista que contém as posições dos vários obstáculos, na forma do predicado $x(L, C)$.
- Uma lista que contém as posições dos vários objetos a apanhar, na forma do predicado $o(L, C)$.

1.1 Exemplo 1

1.1.1 Estado Inicial

```
estado_inicial(  
    e(  
        a(1,2),  
        m(2,2),  
        s(7,2),  
        [x(7,4), x(6,1), x(6,3), x(6,7), x(4,4), x(3,4), x(2,4)], % x's  
        [] % o's  
    )  
).
```

1.1.2 Estado Final

```
estado_final(  
    e(  
        -,  
        m(7,2),  
        s(7,2),  
        [x(7,4), x(6,1), x(6,3), x(6,7), x(4,4), x(3,4), x(2,4)], % x's  
        [] % o's  
    )  
).
```

1.2 Exemplo 2

1.2.1 Estado Inicial

```
estado_inicial(  
    e(  
        a(1,2),  
        m(2,2),  
        s(7,2),  
        [x(7,4), x(6,1), x(6,3), x(6,7), x(4,4), x(3,4), x(2,4)], % x's  
        [o(4,2)] % o's  
    )  
).
```

1.2.2 Estado Final

```
estado_final(  
    e(  
        -/  
        m(7,2),  
        s(7,2),  
        [x(7,4), x(6,1), x(6,3), x(6,7), x(4,4), x(3,4), x(2,4)], % x's  
        [] % o's  
    )  
).
```

1.3 Exemplo 3

1.3.1 Estado Inicial

```
estado_inicial(  
    e(  
        a(1,2),  
        m(2,2),  
        s(7,2),  
        [x(7,3), x(6,1), x(6,3), x(6,7), x(4,4), x(3,4), x(2,4)], % x's  
        [o(5,5)] % o's  
    )  
).
```

1.3.2 Estado Final

```
estado_final(  
    e(  
        -/  
        m(7,2),  
        s(7,2),  
        [x(7,3), x(6,1), x(6,3), x(6,7), x(4,4), x(3,4), x(2,4)], % x's  
        [] % o's  
    )  
).
```

1.4 Exemplo 4

1.4.1 Estado Inicial

```
estado_inicial(  
    e(  
        a(1,2),  
        m(2,2),  
        s(7,5),  
        [x(7,3), x(6,1), x(6,3), x(6,7), x(4,4), x(3,4), x(2,4)], % x's  
        [o(4,2), o(2,6)] % o's  
    )  
).
```

1.4.2 Estado Final

```
estado_final(  
    e(  
        -/  
        m(7,5),  
        s(7,5),  
        [x(7,3), x(6,1), x(6,3), x(6,7), x(4,4), x(3,4), x(2,4)], % x's  
        [] % o's  
    )  
).
```

2 Operadores Transição Estados

Foram utilizados três operadores de transição de estados para conseguir resolver este problema. No caso de empurrar e mover, estes dependem da direção (*cima*, *baixo*, *esq*, *dir*), tendo sido implementados dinamicamente através do predicado `move/5`.

```
op(e(a(La, Ca), m(Lm, Cm), S, Xs, Os), apanhar, e(a(La, Ca), m(Lm, Cm),  
S, Xs, NovoOs), 1) :-  
    pode_apanhar(Lm, Cm, Os),  
    adjacente(La, Ca, Lm, Cm),  
    apanha_o(Lm, Cm, Os, NovoOs).
```

```
op(e(a(La, Ca), m(Lm, Cm), S, Xs, Os), empurrar(D), e(a(Lm, Cm), m(LmN,  
CmN), S, Xs, Os), 1) :-  
    move(D, La, Ca, Lm, Cm),  
    move(D, Lm, Cm, LmN, CmN),  
    lim(LmN, CmN),  
    nao_tem_obstaculos(LmN, CmN, Xs).
```

```
op(e(a(La, Ca), m(Lm, Cm), S, Xs, Os), move(D), e(a(LaN, CaN), m(Lm,  
Cm), S, Xs, Os), 1) :-  
    move(D, La, Ca, LaN, CaN),  
    lim(LaN, CaN),  
    nao_tem_obstaculos(LaN, CaN, Xs),  
    nao_sobreposto(LaN, CaN, Lm, Cm).
```

2.1 Predicados Auxiliares

```
move(dir, L, C1, L, C2) :-  
    C2 is C1 + 1.
```

```
move(esq, L, C1, L, C2) :-  
    C2 is C1 - 1.
```

```
move(cima, L1, C, L2, C) :-  
    L2 is L1 + 1.
```

```
move(baixo, L1, C, L2, C) :-  
    L2 is L1 - 1.
```

```
pode_apanhar(Lm, Cm, Os) :-  
    member(o(Lm, Cm), Os).
```

```
apanha_o(Lm, Cm, Os, NovoOs) :-  
    select(o(Lm, Cm), Os, NovoOs).
```

```
nao_tem_obstaculos(L, C, Xs) :-  
    \+ member(x(L,C), Xs).
```

```
nao_sobreposto(La, Ca, Lm, Cm) :-  
    (La, Ca) \= (Lm, Cm).
```

```
adjacente(La, Ca, Lm, Cm) :-  
    La = Lm,  
    (Ca is Cm + 1);(Ca is Cm - 1), !.
```

```
adjacente(La, Ca, Lm, Cm) :-  
    Ca = Cm,  
    (La is Lm + 1);(La is Lm - 1).
```

3 Algoritmo Pesquisa Não Informada

Ao fazer esta análise dos algoritmos, considerou-se um *branching factor* (b) máximo de 5, sendo o pior caso composto por três `move(D)`, um `apanhar` e um `empurrar(D)`. Sendo este utilizado nos cálculos.

Para a profundidade (d) da melhor solução, considerou-se a profundidade da solução calculada pelo algoritmo A^* , sendo esta a mais fiel para estimar, por ser ótima. Sendo assim, considerou-se $d = 5$ para o exemplo 1, $d = 6$ para o exemplo 2, $d = 20$ para o exemplo 3 e $d = 26$ para o exemplo 4.

Para o cálculo da profundidade máxima (m) considerou-se $+\infty$ para o algoritmo em profundidade, pelo facto de poderem existir ciclos.

Para o limite (l) no caso do algoritmo em profundidade iterativa, também foi considerada a profundidade da solução calculada pelo algoritmo A^* , sendo, portanto, igual ao valor d .

3.1 Nós Visitados

Para o cálculo dos nós visitados consideraram-se as seguintes fórmulas para cada um dos seguintes algoritmos:

- Pesquisa em largura:

$$1 + b + b^2 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1}$$

- Pesquisa em profundidade:

$$b^m$$

- Pesquisa em profundidade limitada:

$$b^l = b^d$$

- Pesquisa em profundidade iterativa:

$$\sum_{i=0}^d (d + 1 - i) \cdot b^i$$

Os seus resultados estimados podem ser vistos na seguinte tabela:

Tabela 1: Estimativa do número de nós visitados por cada algoritmo de pesquisa.

Exemplo	Largura	Profundidade	Prof. Limitada	Prof. Iterativa
1	3906	$+\infty$ ¹	3125	4881
2	19531	$+\infty$ ²	15625	24412
3	$1.19 \cdot 10^{14}$	$+\infty$	5^{20}	$1.49 \cdot 10^{14}$
4	$1.86 \cdot 10^{18}$	$+\infty$	5^{26}	$2.33 \cdot 10^{18}$

¹ Apesar deste valor ser uma estimativa, ao correr o código para este exemplo, foi obtido o valor 6.

² Apesar deste valor ser uma estimativa, ao correr o código para este exemplo, foi obtido o valor 7.

3.2 Nós em Memória

Para o cálculo dos nós em memória consideraram-se as seguintes fórmulas para cada um dos seguintes algoritmos:

- Pesquisa em largura:

$$b^{(d+1)}$$

- Pesquisa em profundidade:

$$b \cdot m$$

- Pesquisa em profundidade limitada:

$$b \cdot l = b \cdot d$$

- Pesquisa em profundidade iterativa:

$$b \cdot d$$

Os seus resultados podem ser vistos na seguinte tabela:

Tabela 2: Número máximo de nós em memória por cada algoritmo de pesquisa.

Exemplo	Largura	Profundidade	Prof. Limitada	Prof. Iterativa
1	5^6	$+\infty$ ¹	25	25
2	5^7	$+\infty$ ²	30	30
3	5^{21}	$+\infty$	100	100
4	5^{27}	$+\infty$	130	130

¹ Apesar deste valor ser uma estimativa, ao correr o código para este exemplo, foi obtido o valor 15.

² Apesar deste valor ser uma estimativa, ao correr o código para este exemplo, foi obtido o valor 19.

3.3 Comparação de Resultados

Ao analisar os resultados da Tabela 1 é possível verificar que a pesquisa em largura e a pesquisa em profundidade iterativa apresentam um número de nós visitados da mesma ordem de grandeza, o que indica que demorariam um intervalo de tempo semelhante para chegarem a uma solução. Já a pesquisa em profundidade limitada, tal como seria de esperar, apresenta um valor ligeiramente inferior às anteriores. Por fim, a pesquisa em profundidade pode entrar em ciclos e nunca encontrar uma solução, o que se traduz num número de nós visitados que pode ser infinito.

Já na Tabela 2 é possível verificar que as pesquisas em profundidade iterativa e em profundidade limitada apresentam um resultado igual e muito melhor do que as outras pesquisas. A pesquisa em largura apresenta um valor muito superior, uma vez que necessita, no máximo, de guardar em memória todos os nós da profundidade da solução. Mais uma vez, a pesquisa em profundidade pode entrar em ciclos, guardando em memória um número infinito de nós.

Sendo assim, podemos descartar a pesquisa em profundidade por não ser ótima nem completa e por poder ter ciclos neste problema. Em relação à profundidade limitada, apesar de parecer boa com estes resultados, não é ótima e é necessário conhecer *à priori* a profundidade a que está a solução, de modo a realizar um corte útil. A decisão final ficou, portanto, entre a pesquisa em largura e a pesquisa em profundidade iterativa. Como este problema tem uma dimensão bastante grande, a pesquisa em largura necessita de guardar um número muito elevado de nós simultaneamente em memória, ao contrário da pesquisa em profundidade iterativa que tem um valor muito pequeno. Para além do número de nós em memória ser bastante decisivo, achamos que o número de nós visitados não representou uma diferença muito significativa.

Concluimos, assim, que a pesquisa em profundidade iterativa é a melhor para este problema.

4 Conclusões Pesquisa Não Informada

4.1 Exemplo 1

4.1.1 Solução

1. empurrar(cima)
2. empurrar(cima)
3. empurrar(cima)
4. empurrar(cima)
5. empurrar(cima)

4.1.2 Número Estados Visitados

Para chegar à solução acima, foram visitados 187 nós.

4.1.3 Número Estados Memória

Como não está a ser feito o fecho, o número de estados em memória quando se atingiu esta solução foi 15.

4.2 Exemplo 2

4.2.1 Solução

1. empurrar(cima)
2. empurrar(cima)
3. apanhar
4. empurrar(cima)
5. empurrar(cima)
6. empurrar(cima)

4.2.2 Número Estados Visitados

Para chegar à solução acima, foram visitados 614 nós.

4.2.3 Número Estados Memória

Como não está a ser feito o fecho, o número de estados em memória quando se atingiu esta solução foi 19.

4.3 Exemplo 3

O programa não conseguiu terminar, ocorrendo sempre um *global stack overflow* – mesmo com `GLOBALSZ = 2000000` –, pelo que não foi possível encontrar uma solução.

4.4 Exemplo 4

O programa não conseguiu terminar, ocorrendo sempre um *global stack overflow* – mesmo com `GLOBALSZ = 2000000` –, pelo que não foi possível encontrar uma solução.

5 Heurísticas

Para os algoritmos de pesquisa informada foram realizadas duas heurísticas, uma mais simples e uma mais complexa, para tentar demonstrar o seu impacto.

5.1 Heurística 1

A primeira heurística é a soma do número de objetos por apanhar com a distância de *Manhattan* da máquina à saída. Esta é admissível porque cada objeto precisa pelo menos de uma ação para ser apanhado, pelo que o número de objetos restantes é sempre um limite inferior do custo real. Para além disso, a distância de *Manhattan* à saída será sempre menor ou igual ao número de movimentos para esta, pelo facto de também haver obstáculos. Desta forma, a heurística nunca sobrestima o verdadeiro custo de chegar ao objetivo.

Em Prolog, esta heurística foi implementada da seguinte maneira:

```
h1(e(a(La, Ca), m(Lm, Cm), s(Ls, Cs), _, Os), H) :-  
    % Número de objetos por apanhar  
    length(Os, Nobj),  
    % Distância da máquina à saída  
    dist_manhattan((Lm, Cm), (Ls, Cs), Dms),  
    H is Dms + Nobj.
```

5.2 Heurística 2

A segunda heurística representa a soma da distância de *Manhattan* do agente a uma posição adjacente à máquina, com o número de objetos por apanhar e o caminho maior existente para chegar à saída, passando por um objeto (se existir).

Esta heurística é admissível porque nunca sobrestima o custo real, analisando cada componente:

- **Distância do agente à máquina** ($D - 1$): o agente terá sempre de percorrer pelo menos $D - 1$ passos para ficar adjacente à máquina e a poder empurrar, sendo portanto um limite inferior do custo real.
- **Número de objetos**: cada objeto precisa de pelo menos uma ação de apanhar, pelo que o número de objetos restantes é um limite inferior do custo real.
- **Rota da máquina, objeto, saída**: representa a maior distância para a máquina percorrer desde a sua posição atual, passando por um objeto, até à saída. Para qualquer objeto no caminho real da máquina, o custo real é sempre maior ou igual ao caminho direto máquina $\rightarrow$ objeto $\rightarrow$ saída, independentemente dos outros objetos.

Como isto se verifica para qualquer objeto, verifica-se também para o máximo, tratando-se, assim, de um limite inferior do custo real.

Como cada componente é individualmente admissível, a sua soma também nunca sobrepõe o custo real, garantindo assim a admissibilidade da heurística.

Em Prolog, esta heurística foi implementada da seguinte maneira:

```
h2(e(a(La, Ca), m(Lm, Cm), s(Ls, Cs), _, Os), H) :-  
    % Distância do agente à máquina  
    dist_manhattan((La, Ca), (Lm, Cm), D),  
    Dam is D - 1,  
    % Número de objetos por apanhar  
    length(Os, Nobj),  
    rota_maxima_MOS((Lm, Cm), (Ls, Cs), Os, Dro),  
    H is Dam + Nobj + Dro.
```

5.3 Predicados Auxiliares

```
dist_manhattan((L1, C1), (L2, C2), D) :-  
    D is abs(L1 - L2) + abs(C1 - C2).
```

```
% Sem objetos: máquina vai diretamente à saída  
rota_maxima_MOS(M, S, [], D) :-  
    dist_manhattan(M, S, D).
```

```
% Com objetos: escolhe o maior valor de M -> O -> S  
rota_maxima_MOS(M, S, Os, Dmax) :-  
    Os \= [],  
    findall(  
        D,  
        (  
            member(o(Lo,Co), Os),  
            dist_manhattan(M, (Lo,Co), Dmo),  
            dist_manhattan((Lo,Co), S, Dos),  
            D is Dmo + Dos  
        ),  
        Ds  
    ),  
    max_list(Ds, Dmax).
```

6 Algoritmos Pesquisa Informada

O código em Prolog dos algoritmos encontra-se no ficheiro `pi.pl`. Já o código do problema, com as heurísticas, encontra-se no ficheiro `problema.pl`.

7 Conclusões Pesquisa Informada

Para o cálculo do número de nós em memória, foi necessário somar o número de nós na lista aberta com os nós da lista fechada, num determinado instante.

7.1 Heurística 1

Tabela 3: Número de nós visitados e em memória para a primeira heurística.

Exemplos	Algoritmo Greedy		Algoritmo A*	
	Nós Visitados	Nós Memória	Nós Visitados	Nós Memória
1	6	20	6	20
2	7	25	10	34
3	1758	1806	3164	3871
4	3587	3641	14927	17695

7.1.1 Solução do Exemplo 1

Para ambos os algoritmos, a solução encontrada foi a seguinte:

1. empurrar(cima)
2. empurrar(cima)
3. empurrar(cima)
4. empurrar(cima)
5. empurrar(cima)

7.1.2 Solução do Exemplo 2

Para ambos os algoritmos, a solução encontrada foi a seguinte:

1. empurrar(cima)
2. empurrar(cima)
3. apanhar
4. empurrar(cima)
5. empurrar(cima)
6. empurrar(cima)

7.1.3 Solução do Exemplo 3

Para ambos os algoritmos, a solução encontrada foi a seguinte:

1. empurrar(cima)
2. empurrar(cima)
3. empurrar(cima)
4. move(esq)
5. move(cima)
6. empurrar(dir)
7. empurrar(dir)
8. empurrar(dir)
9. apanhar
10. move(cima)
11. move(dir)
12. move(dir)
13. move(baixo)
14. empurrar(esq)
15. empurrar(esq)
16. empurrar(esq)
17. move(baixo)
18. move(esq)
19. empurrar(cima)
20. empurrar(cima)

7.1.4 Solução do Exemplo 4

O algoritmo A* obteve a seguinte solução:

1. empurrar(cima)
2. empurrar(cima)
3. apanhar
4. empurrar(cima)
5. move(esq)
6. move(cima)
7. empurrar(dir)
8. empurrar(dir)
9. empurrar(dir)
10. empurrar(dir)
11. move(cima)
12. move(dir)

13. empurrar(baixo)
14. empurrar(baixo)
15. empurrar(baixo)
16. apanhar
17. move(dir)
18. move(baixo)
19. empurrar(esq)
20. move(baixo)
21. move(esq)
22. empurrar(cima)
23. empurrar(cima)
24. empurrar(cima)
25. empurrar(cima)
26. empurrar(cima)

O algoritmo *Greedy* obteve a seguinte solução:

1. empurrar(cima)
2. empurrar(cima)
3. apanhar
4. empurrar(cima)
5. move(esq)
6. move(cima)
7. empurrar(dir)
8. empurrar(dir)
9. empurrar(dir)
10. move(cima)
11. move(dir)
12. move(dir)
13. move(baixo)
14. move(baixo)
15. move(esq)
16. empurrar(cima)
17. move(esq)
18. move(cima)
19. empurrar(dir)
20. move(cima)
21. move(dir)
22. empurrar(baixo)
23. empurrar(baixo)

24. empurrar(baixo)
25. empurrar(baixo)
26. apanhar
27. move(dir)
28. move(baixo)
29. empurrar(esq)
30. move(baixo)
31. move(esq)
32. empurrar(cima)
33. empurrar(cima)
34. empurrar(cima)
35. empurrar(cima)
36. empurrar(cima)

7.2 Heurística 2

Tabela 4: Número de nós visitados e em memória para a segunda heurística.

Exemplos	Algoritmo Greedy		Algoritmo A*	
	Nós Visitados	Nós Memória	Nós Visitados	Nós Memória
1	6	20	6	20
2	7	25	7	25
3	29	85	588	1097
4	478	861	2642	4178

7.2.1 Solução do Exemplo 1

Para ambos os algoritmos, a solução encontrada foi a seguinte:

1. empurrar(cima)
2. empurrar(cima)
3. empurrar(cima)
4. empurrar(cima)
5. empurrar(cima)

7.2.2 Solução do Exemplo 2

Para ambos os algoritmos, a solução encontrada foi a seguinte:

1. empurrar(cima)
2. empurrar(cima)
3. apanhar
4. empurrar(cima)
5. empurrar(cima)
6. empurrar(cima)

7.2.3 Solução do Exemplo 3

Para ambos os algoritmos, a solução encontrada foi a seguinte:

1. empurrar(cima)
2. empurrar(cima)
3. empurrar(cima)
4. move(esq)
5. move(cima)
6. empurrar(dir)
7. empurrar(dir)
8. empurrar(dir)
9. apanhar
10. move(cima)
11. move(dir)
12. move(dir)
13. move(baixo)
14. empurrar(esq)
15. empurrar(esq)
16. empurrar(esq)
17. move(baixo)
18. move(esq)
19. empurrar(cima)
20. empurrar(cima)

7.2.4 Solução do Exemplo 4

O algoritmo A* obteve a seguinte solução:

1. empurrar(cima)
2. empurrar(cima)
3. apanhar
4. empurrar(cima)
5. move(esq)
6. move(cima)
7. empurrar(dir)
8. empurrar(dir)
9. empurrar(dir)
10. empurrar(dir)
11. move(cima)
12. move(dir)
13. empurrar(baixo)
14. empurrar(baixo)
15. empurrar(baixo)
16. apanhar
17. move(dir)
18. move(baixo)
19. empurrar(esq)
20. move(baixo)
21. move(esq)
22. empurrar(cima)
23. empurrar(cima)
24. empurrar(cima)
25. empurrar(cima)
26. empurrar(cima)

O algoritmo *Greedy* obteve a seguinte solução:

1. empurrar(cima)
2. empurrar(cima)
3. apanhar
4. move(esq)
5. move(cima)
6. empurrar(dir)
7. move(cima)
8. move(dir)

```
9. empurrar(baixo)
10. empurrar(baixo)
11. move(esq)
12. move(baixo)
13. move(baixo)
14. move(dir)
15. empurrar(cima)
16. empurrar(cima)
17. empurrar(cima)
18. move(esq)
19. move(cima)
20. empurrar(dir)
21. empurrar(dir)
22. empurrar(dir)
23. move(cima)
24. move(dir)
25. empurrar(baixo)
26. empurrar(baixo)
27. empurrar(baixo)
28. apanhar
29. move(dir)
30. move(baixo)
31. empurrar(esq)
32. move(baixo)
33. move(esq)
34. empurrar(cima)
35. empurrar(cima)
36. empurrar(cima)
37. empurrar(cima)
38. empurrar(cima)
```

Instruções para Correr o Problema

No ficheiro `problema.pl` basta apenas trocar o estado inicial e final, para o do tabuleiro que se quer testar (e.g. `estado_inicial_1(E)` para `estado_inicial_2(E)`). A mesma coisa aplica-se para as heurísticas (e.g. `h1(E, H)` para `h2(E, H)`).

Para correr o problema com o algoritmo de pesquisa não informada, deve consultar o ficheiro `pni.pl` e correr `pesquisa(problema)`. No caso da pesquisa informada, `pi.pl`, é correr `pesquisa(problema, a)` ou `pesquisa(problema, g)`.